

Experiment No.11
Case study on compiler construction frameworks.
Date of Performance:
Date of Submission:



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Aim: Case study on compiler construction frameworks.

Objective: To develop an understanding about compiler construction frameworks mostly used in companies such as apple, google, Cray etc.

Theory:

Abstract:

The paper discusses LLVM, a compiler framework supporting lifelong program analysis and transformation at compile-time, link-time, run-time, and offline. LLVM uses Static Single Assignment (SSA) form with features like a language-independent type system, typed address arithmetic, and an efficient exception handling mechanism. The framework offers key capabilities crucial for practical program analysis and transformation. The design is evaluated in terms of representation size, effectiveness, compiler performance, and benefits in challenging compiler problems. According to the paper, LLVM provides unique capabilities not found in existing compilation approaches.

Introduction

At heart, LLVM is a library for programmatically creating machine-native code. A developer uses the API to generate instructions in a format called an intermediate representation, or IR. LLVM can then compile the IR into a standalone binary or perform a JIT (just-in-time) compilation on the code to run in the context of another program, such as an interpreter or runtime for the language.

LLVM's APIs provide primitives for developing many common structures and patterns found in programming languages. For example, almost every language has the concept of a function and of a global variable, and many have coroutines and C foreign-function interfaces. LLVM has functions and global variables as standard elements in its IR, and it has metaphors for creating coroutines and interfacing with C libraries.

Program Representation

The LLVM compiler framework represents programs using a common low-level code representation in Static Single Assignment (SSA) form. It features a language-independent type system, includes an instruction for typed address arithmetic, and provides a simple mechanism for efficient exception handling. The representation is modular, extensible, and used consistently throughout the compilation process, offering key capabilities for program analysis and transformation.

Overview of the LLVM Instruction Set:

CSL601: System Programming and Compiler Construction Laboratory



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

The LLVM instruction set, integral to the LLVM Intermediate Representation (IR), serves as a pivotal component in the compilation process, bridging the gap between high-level source code and low-level machine code. Central to its design is the utilization of Static Single Assignment (SSA) form, where variables are assigned only once, providing a structured representation of computations. The instruction set encompasses a wide array of typed instructions that cater to various data types, including integers, floating-point numbers, and pointers. Fundamental arithmetic and logical operations, such as addition, subtraction, multiplication, and logical AND, OR, and XOR, are supported. Memory operations, represented by load and store instructions, facilitate efficient data transfer between registers and memory.

Control flow in LLVM is managed through instructions like conditional branches (br) and unconditional branches (br label). Phi nodes, essential for maintaining the integrity of the SSA form, are employed at merge points in control flow. Function calls are represented through call and invoke instructions, with the latter incorporating exception handling information. Notably, LLVM introduces an instruction for typed address arithmetic, allowing precise manipulation of memory addresses based on variable types.

The instruction set extends its capabilities to include vector operations, supporting SIMD (Single Instruction, Multiple Data) processing for parallel computation. Additionally, atomic instructions address concurrent programming needs, ensuring consistency in shared memory environments. Metadata can be associated with instructions, providing supplementary information for optimization and debugging purposes. Intrinsic functions within the instruction set allow for low-level operations, enabling interactions with specific hardware features that may not be expressible directly in high-level source code.

In essence, the LLVM instruction set offers a comprehensive suite of typed instructions, each tailored to represent specific facets of computations and control flow. This versatility facilitates sophisticated program analysis, optimization, and transformation throughout the compilation pipeline.

How to work with LLVM

The typical way to work with LLVM is via code in a language that supports LLVM's libraries.

Two common language choices are C and C++. Many LLVM developers default to one of those two for good reasons:

LLVM itself is written in C++.

LLVM's APIs are available in C and C++ incarnations.

CSL601: System Programming and Compiler Construction Laboratory



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Much language development tends to happen with C/C++ as a base.

Still, those two languages are not the only choices. Many languages can call natively into C libraries, so it's theoretically possible to perform LLVM development with any such language. But it helps to have an actual library in the language that elegantly wraps LLVM's APIs. Fortunately, many languages and language runtimes have such libraries, including C#/.NET/Mono, Rust, Haskell, OCAML, Node.js, Go, and Python.

One caveat is that some of the language bindings to LLVM may be less complete than others. With Python, for example, many choices cropped up over time, varying in completeness and utility, with a top contender emerging.

llvmlite, developed by the team that creates Numba, has emerged as the current contender for working with LLVM in Python. It implements only a subset of LLVM's functionality, as dictated by the needs of the Numba project. But that subset provides the vast majority of what LLVM users need. To that end, llvmlite is generally the best choice for working with LLVM in Python.

The LLVM project has its own set of bindings to LLVM's C API, but they are currently not maintained.

llvmpy, the first popular Python binding for LLVM, fell out of maintenance in 2015. That's bad for any software project, but worse when working with LLVM, given the number of changes that come along in each new edition.

llvmcpy aims to bring the Python bindings for the C library up to date, keep them updated in an automated way, and make them accessible using Python's native idioms. llvmcpy can currently do some rudimentary work with the LLVM APIs, but it has not been updated since 2019.

If you're curious about how to use LLVM libraries to build a language, LLVM's creators have a tutorial, using either C++ or OCAML, that steps you through creating a simple language called Kaleidoscope. It's since been ported to other languages:

Haskell: A direct port of the original tutorial.

Python: One such port follows the tutorial closely, while the other is a more ambitious rewrite with an interactive command line. Both of those use llvmlite as the bindings to LLVM.

Rust and Swift: It seemed inevitable we'd get ports of the tutorial to two of the languages that LLVM helped bring into existence.



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Finally, the LLVM tutorial is also available in human languages. It has been translated into Chinese, using the original C++ and Python.

Conclusion: Compiler construction frameworks provide tools and libraries for building compilers and interpreters. Key features include lexical and syntax analysis, abstract syntax trees, semantic analysis, intermediate representation, code generation, optimization, error handling, extensibility, toolchain integration, documentation, and cross-platform compatibility. These frameworks simplify the development process and allow developers to focus on language design and optimization.



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering
